KU LEUVEN kulak

Groep Wetenschap & Technologie Kulak
Teamopdracht

Werken met Git en GitHub vanuit VS Code

Academiejaar 2026 – 2027

Stijn Rebry

Inhoudsopgave

1 Software installeren, account activeren **3**
- 1.1 VS Code installeren . . . 3
- 1.2 GitHub-account maken . . . 3
- 1.3 Account koppelen aan GitHub Education . . . 4
- 1.4 Git installeren . . . 4

2 Local repository, commits **4**
- 2.1 Lokale map maken . . . 4
- 2.2 Lokale map openen in VS Code . . . 4
- 2.3 Lokale repository initialiseren . . . 5
- 2.4 Eerste commit . . . 5
- 2.5 Aanpassingen doorvoeren aan een bestand . . . 5
- 2.6 Bestanden toevoegen en verwijderen . . . 6
- 2.7 Stage – selecteren van aanpassingen voor commit . . . 6
- 2.8 Terminologie . . . 7

3 Repository publiceren op GitHub **7**
- 3.1 Repository publiceren vanuit VS Code . . . 7
- 3.2 Repository bekijken op GitHub . . . 7
- 3.3 Lokale wijzigingen maken . . . 7
- 3.4 Push uitvoeren . . . 8
- 3.5 Wijzigingen rechtstreeks op GitHub aanbrengen . . . 8
- 3.6 Pull uitvoeren . . . 8
- 3.7 Terminologie . . . 8

4 Werken met branches **9**
- 4.1 De huidige branch bekijken . . . 9
- 4.2 Een nieuwe branch maken . . . 9
- 4.3 Een wijziging maken op de nieuwe branch, local repository . . . 9
- 4.4 Een nieuwe branch op de GitHub-repository . . . 10
- 4.5 Fetch – online informatie ophalen . . . 10
- 4.6 Pull – Branches van GitHub halen . . . 10
- 4.7 Push – Branches publiceren op GitHub . . . 11
- 4.8 Terminologie . . . 11

5 Samenvoegen van branches: merge **11**
- 5.1 Lokale merge van nieuwe bestanden . . . 11
- 5.2 Remote merge van nieuwe bestanden . . . 12
- 5.3 Merge zonder conflicten . . . 12
- 5.4 Een merge met conflict . . . 13
- 5.5 Terminologie . . . 13

6 Starten van een bestaande repository **13**
- 6.1 Fork – kopie van een repository van een andere eigenaar . . . 13
- 6.2 Clone – een online repository ophalen . . . 14
- 6.3 Terminologie . . . 15

7 Werkverdeling: issues en assignees **15**
7.1 Een issue en branch aanmaken . . . 15
7.2 Een assignee toekennen . . . 15
7.3 De wijziging uitvoeren . . . 16
7.4 Het issue sluiten . . . 16
7.5 Terminologie . . . 16

8 Projectplanning met GitHub Projects **17**
8.1 Een project aanmaken en delen . . . 17
8.2 Issues aan het project toevoegen . . . 17
8.3 De voortgang bijhouden . . . 17
8.4 Deadlines toevoegen en zichtbaar maken . . . 17

9 De teamopdracht: repository, planning en workflow **18**
9.1 Starten in je teamrepository . . . 18
9.2 Bestanden uit sessies 1 tot en met 3 . . . 18
9.3 Branches en verantwoordelijkheden . . . 19
9.4 Het projectbord en de planning afspreken . . . 19
9.5 Taken vastleggen en verdelen . . . 20
9.6 Opdracht voor de vrijdagsessie: bestaand werk inbrengen . . . 20
9.7 Van bijdrage tot afgewerkte taak . . . 21
9.8 Wanneer gaat het oefeningenpakket naar main? . . . 21
9.9 Nieuwe opgaven ophalen . . . 21
9.10 De voortgang samen opvolgen . . . 21
9.11 Je bijdrage zichtbaar maken . . . 22

Inleiding

Het lokale programma Git en het online platform GitHub zijn hulpmiddelen om versies van bestanden bij te houden en met meerdere personen samen te werken aan een project. Oorspronkelijk werden ze ontwikkeld voor softwareontwikkeling, maar ondertussen worden ze ook gebruikt voor verslagen, onderzoeksprojecten, websites, presentaties, data-analyse en vele andere toepassingen waarbij meerdere versies van bestanden ontstaan.

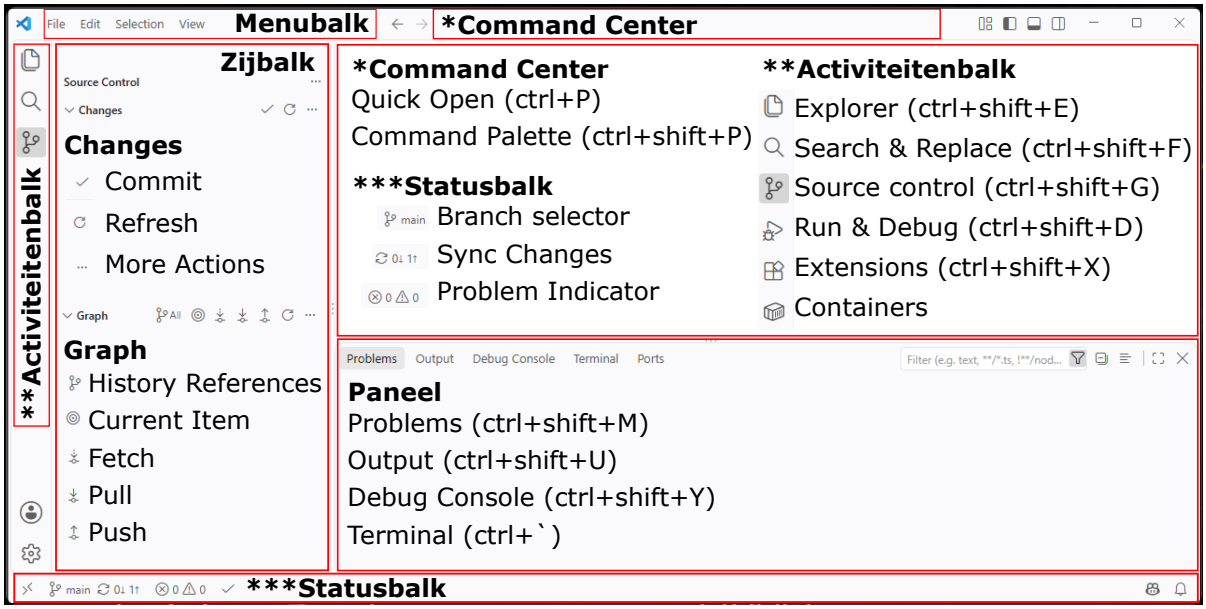
De basisidee is eenvoudig. In plaats van telkens bestanden op te slaan onder een andere naam zoals `versie1`, `versie2` of `eindversie_definitief_echt_definitief`, bewaart Git een gedocumenteerde geschiedenis van alle wijzigingen. Op elk moment kan worden nagegaan wie een wijziging heeft uitgevoerd, wanneer die wijziging gebeurde en hoe een eerdere versie van een bestand eruitzag.

Git bestaat essentieel uit honderden tekstcommando's die via de commandoregel kunnen worden uitgevoerd en biedt talloze geavanceerde mogelijkheden voor versiebeheer, samenwerking, testen en automatisering. In de praktijk werken veel gebruikers niet rechtstreeks met deze commando's, maar via een grafische gebruikersinterface die de meest gebruikte Git-functies toegankelijk maakt met knoppen, menu's en overzichtsvensters.

GitHub is een online platform dat Git-repositories kan bewaren en delen. Het voegt bovendien extra functionaliteit toe voor samenwerking, zoals issues, taakverdeling en projectbeheer. GitHub zelf vervangt Git niet, maar bouwt er bovenop voort. Er zijn nog andere manieren om GitHub aan te sturen, maar deze handleiding gebruikt de webinterface.

Git kan vanuit verschillende programma's worden gebruikt. In deze handleiding kiezen we voor één werkwijze: Git aansturen vanuit Visual Studio Code (VS Code). Waar mogelijk gebruiken we de grafische interface van VS Code en GitHub. Slechts af en toe zullen we een opdracht uit het `Command Palette` gebruiken. Daardoor kunnen we ons concentreren op de onderliggende ideeën zonder eerst een lange lijst Git-commando's te moeten leren.

Na het doorlopen van de installatie-instructies, starten we met het introduceren van versiebeheer op de eigen lokale computer (local repository, commit – sectie 2), om die gegevens vervolgens te publiceren op GitHub (remote repository, push, pull – sectie 3). Daarna onderzoeken we hoe verschillende versies van hetzelfde project naast elkaar kunnen bestaan (branches, fetch – sectie 4) en hoe deze versies daarna



Figuur 1: Benaming van de belangrijkste schermonderdelen in VS Code met betrekking tot Git

opnieuw kunnen worden gecombineerd (*merge* – sectie 5). In een groter project waar meerderen samenwerken, is het belangrijk het overzicht te bewaren van wat er nog te gebeuren staat en wie wat doet (*issues*, *assignees* – sectie 7). Ten slotte passen we deze workflow toe op de teamopdracht van het vak (sectie 9).

Na het doorwerken van deze handleiding beheers je de basisworkflow die ook in professionele software- en onderzoeksprojecten wordt gebruikt, om de bijdragen van verschillende personen gecontroleerd samen te voegen.

1 Software installeren, account activeren

1.1 VS Code installeren

- Download de laatste versie van VS Code via `code.visualstudio.com`
- Selecteer de gewenste taal, aanvaard de gebruiksovereenkomst en klik door de volgende dialoogschermen: tenzij je andere voorkeuren hebt, zijn de standaardinstellingen goed.
- Klik op `Install`.

1.2 GitHub-account maken

- Ga naar de webpagina `github.com` om een account te maken.
- Vul je KU Leuven-e-mailadres in en klik op `Sign up for GitHub`.
- Kies een veilig paswoord, hergebruik geen paswoorden.
- Kies een username en klik op `Create account`.
- Je krijgt een e-mail met een beveiligingscode die je moet invullen op de volgende pagina. Daarna is je account actief.

1.3 Account koppelen aan GitHub Education

- Log in op GitHub.
- Open de pagina `github.com/settings/education/benefits` en klik op `Start an application`.
- Op basis van je e-mailadres herkent GitHub jouw instelling als Catholic University of Leuven, klik `Select this school`, `Share location` en `Continue`.
- Stuur een foto van je studentenkaart om toegang te vragen en volg eventuele verdere instructies.

1.4 Git installeren

- Windows-gebruikers die niet eerder Git gebruikten, moeten dit programma eerst nog installeren. Download het installatieprogramma op `git-scm.com`.
- Start de installatie, aanvaard de gebruiksovereenkomst en klik door de volgende dialoogschermen: tenzij je andere voorkeuren hebt, zijn de standaardinstellingen goed.
- Tot slot is het nodig om je gebruikersinformatie te configureren. Open de terminal in VS Code (`ctrl+‘`) en voer daarin volgende commando's uit:

```
git config --global user.name "Voornaam Naam"
git config --global user.email "jouw-e-mail@student.kuleuven.be"
```

2 Local repository, commits

2.1 Lokale map maken

Git-projecten zullen we opslaan in een lokale map, *niet* in de cloud (zoals OneDrive, Google Drive, Dropbox, ...). Git schrijft immers voortdurend bestanden weg en services als OneDrive zullen diezelfde bestanden tegelijk synchroniseren, wat conflicten, corrupte bestanden of prestatieproblemen kan veroorzaken.

- Open de Windows Bestandsverkenner (Windows Explorer, `windows+E`).
- Typ in de adresbalk "`C:\`" en druk op enter.
- Klik links in de knoppenbalk op `Nieuw`, `Map`.
- Geef de map de naam "`Projecten`".
- Maak nog een map aan in `C:\Projecten` met de naam "`gittest`".

Het hele punt van Git is natuurlijk wel om ook een kopie van de repository in de cloud te bewaren, als backup en om te delen met anderen, maar dat regelen we later in deze handleiding.

2.2 Lokale map openen in VS Code

Een project bestaat typisch niet uit één bestand, maar uit heel wat bestanden die worden verzameld in een map. We openen in VS Code de lokale map die net is gemaakt.

- Als je al andere bestanden hebt geopend in VS Code, open dan een nieuw venster, `File`, `New Window` (`ctrl+shift+N`).
- Open de map `C:\Projecten\gittest` in VS Code via `File`, `Open Folder`.
- Maak meteen een nieuw tekstbestand aan (`ctrl+N`): `File`, `New Text File`. Typ hierin enkel je naam (lijn 1) en opleiding (lijn 2):

```
Naam: jouw voornaam en jouw naam
Opleiding: jouw opleiding
```

- Sla dit bestand op (`ctrl+S`): `File`, `Save` met als naam "`gegevens.txt`".

2.3 Lokale repository initialiseren

Op dit moment is de werkmap een gewone bestandsmap zonder versiecontrole. Om die te activeren, maken we er een Git-repository van.

- Klik in VS Code op het icoon `Source Control` (derde icoon links in de activiteitenbalk, zie 1, `ctrl+shift+G`).
- Klik op `Manage Workspace Trust` en daarna op `Trust`, sluit het venster.
  - `Trust`: VS Code beschouwt de map als veilig en schakelt alle functionaliteit in, waaronder Git-integratie, extensies en het uitvoeren van code.
  - `Don't Trust`: VS Code opent de map in `Restricted Mode`. Bestanden kunnen nog worden bekeken en aangepast, maar bepaalde functionaliteit wordt uit veiligheidsoverwegingen uitgeschakeld.
- Klik op `Initialize Repository` om van de werkmap een repository te maken.
  - Je ziet dat het bestand `gegevens.txt` wordt vermeld onder `Changes`. Onderaan in de zijbalk is de `Graph` nog leeg, hier verschijnt later de inhoud van de repository.

2.4 Eerste commit

Nu kunnen we het eerste bestand met een commit in de versiegeschiedenis opnemen.

- Tik in het vak `Message` (onder `Changes` en boven de `Commit`-knop) een beschrijving van de aanpassingen, in dit geval bijvoorbeeld "Initiële inhoud".
  - Het invoeren van een commit-message is absoluut noodzakelijk. Vergeet je dit, dan opent er een bestandje waarin je de commit-message nog kan ingeven op de eerste lijn, waarna je dit bestand afsluit.
  - Vul je geen commit-message in het invulvak of in het bestandje, dan wordt de commit afgebroken.
- Klik op `Commit`. Een pop-up vraagt "stage all changes and commit?", kies voor `Yes`. De rol van stage komt aan bod in sectie 2.7.
- Klik op `Initiële inhoud` onder `Graph`.
  - Het bestand `gegevens.txt` verschijnt met daarnaast de letter `A` (added file).

2.5 Aanpassingen doorvoeren aan een bestand

Nu zullen we het bestand `gegevens.txt` aanpassen en de aanpassingen doorvoeren naar de repository.

- Voeg aan het bestand `gegevens.txt` een derde lijn toe met daarop `Woonplaats: Gallifrey` (dat is wellicht fout, maar corrigeren we later). Sla het bestand op (`ctrl+S`).
- De bestandsnaam `gegevens.txt` is nu *drie* keer te zien in de VS Code-omgeving:
  - Het bestand in het editorvenster, met de laatste aanpassingen, is opgeslagen maar zit nog niet in de repository.
  - Klik op `gegevens.txt` onder `Changes`. Een nieuw venster opent in de editor met daarin de verschillen tussen de recentste versie van het bestand en de versie die in de repository staat. De toegevoegde lijn wordt groen gemarkeerd.
  - Klik op `gegevens.txt` onder `Graph`. Een nieuw venster opent in de editor met daarin de versie van het bestand zoals die is opgeslagen in de meest recente commit van de repository. Deze bevat nog maar twee lijnen.

Git bewaart dus zowel de huidige bestanden in je werkmap als eerdere versies in de repository.

- Typ opnieuw een beschrijving van de aanpassingen in het vak `Message` onder `Changes`, bijvoorbeeld "Woonplaats toegevoegd", klik op `Commit`, klik op `Yes` in de pop-up.
  - De `Graph` toont nu twee commits, met de meest recente bovenaan en telkens met het bestand `gegevens.txt`. Door erop te klikken kan je de versies vergelijken.
  - Naast het bestand in de meest recente commit staat de letter `M` (modified file).

  Met een commit leg je een versie van je project permanent vast in de repository, de message laat toe om later gericht een bepaalde versie terug te vinden. De repository bevat dus alle versies van je project, ook al zijn bestanden aangepast of zelfs verwijderd uit de werkmap.

2.6 Bestanden toevoegen en verwijderen

Nu zullen we nieuwe bestanden toevoegen en een bestaand bestand verwijderen.

- Maak twee nieuwe bestanden (`ctrl+N`) met als namen `brol.txt` (willekeurige inhoud) en `steden.txt` (bevat vier lijnen met daarop respectievelijk `Brugge`, `Kortrijk`, `Leuven` en `Poelkapelle`). Sla deze bestanden op (`ctrl+S`) en voer een commit uit, klik op `Yes` in de pop-up.
  - In de `Graph` staan inmiddels drie commits, met daarbij telkens enkel de bestanden die in die commit zijn toegevoegd (`A`) of aangepast (`M`).
- Verwijder via de `Explorer` in VS Code (`ctrl+shift+E`, bovenste knop in de activiteitenbalk, zie figuur 1) het bestand `brol.txt` uit de map (selecteren + `Delete`), bevestig met `Move to recycle bin`.
- Schrap het gehucht `Poelkapelle` uit de opsomming in `steden.txt`, sla het bestand op en commit de verwijderingen, klik op `Yes` in de pop-up.
  - In `Source Control`, `Graph` wordt het bestand `brol.txt` gemarkeerd met de letter `D` (deleted file).
  - Het bestand `steden.txt` staat onder de recentste commit in de `Graph` gemarkeerd met de letter `M` (modified file). Openen hiervan in de editor toont de geschrapte lijn in het rood gemarkeerd.

Ook al is de informatie uit de map of uit een bestand verwijderd (zie `Explorer`), toch verdwijnt er geen informatie uit de repository (zie `Source Control`).

2.7 Stage – selecteren van aanpassingen voor commit

Hierboven werd een stap overgeslagen bij het committen. In een realistische workflow komt tussen opslaan en committen van aanpassingen nog het stagen. Niet alle wijzigingen die tegelijk in de werkmap aanwezig zijn, horen immers noodzakelijk in dezelfde commit thuis. Bekijk volgend voorbeeld: bij het schrijven van het eindverslag wordt een figuur gemaakt en geïnterpreteerd in de tekst.

- Het maken van de figuur, verzorgen van de assen en de legende gebeurt in een `Python`-notebook waar alle geschreven code wordt getest en geïllustreerd.
- Het invoegen van de figuur gebeurt in een LaTeX-file. Er wordt een bijschrift gemaakt dat toelichting geeft bij de verschillende elementen op de figuur en een verwijzing in de tekst geschreven naar deze figuur.
- Tijdens het werken worden beide bestanden afwisselend aangepast: de figuur wordt verbeterd, daarna wordt het bijschrift aangepast, vervolgens wordt opnieuw aan de figuur gewerkt, enzovoort.
- Deze aanpassingen kunnen dan in twee commits worden doorgevoerd in de repository: één voor alles wat met het Python-notebook gebeurt en een tweede voor de aanvullingen in het verslag.

Hiervoor gebruikt `Git` de stage, wat leidt tot de volgende meer geavanceerde workflow Save – Stage – Commit. In het vervolg van deze tekst zullen we steeds alle aanpassingen tegelijk stagen, klik in de pop-up dus steeds op `Yes`, of eenmalig op `Always`.

2.8 Terminologie

Hiermee zijn alvast vier belangrijke begrippen geïntroduceerd:

- *Werkmap*: de lokale map waarin de bestanden van het project staan.
- *Stage*: selectie van wijzigingen die in de volgende commit zullen terechtkomen.
- *Commit*: een opgeslagen versie van de werkmap op één bepaald moment.
- *Repository*: het geheel van alle commits en dus alle aanpassingen en versies van elk bestand.

3 Repository publiceren op GitHub

Tot nu toe staat de repository alleen op de eigen computer. Dat is niet erg veilig, je kan informatie verliezen door een defect of diefstal van je laptop. Het is op die manier ook niet mogelijk om samen te werken met anderen. Daarom zullen we een kopie van de repository online bewaren, op GitHub.

3.1 Repository publiceren vanuit VS Code

De lokale repository `gittest` zullen we nu publiceren op GitHub.

- Klik in `Source Control` op `Publish Branch`. Deze knop staat er enkel als er geen hangende aanpassingen meer zijn die op een commit wachten.
- Vervolg het proces om in te loggen met de account die gelinkt is aan je KU Leuven-e-mailadres.
- Kies `Publish to github Private Repository`.
  - Een private repository is enkel zichtbaar voor de eigenaar en voor personen die expliciet toegang krijgen.
  - Een public repository is zichtbaar voor iedereen.
- Meld eventueel aan met je GitHub-account.
- Bevestig de publicatie.
  - Als VS Code vraagt om regelmatig `git fetch` uit te voeren, klik je op `No`. Wat dit precies doet, komt later aan bod (sectie 4.5).

3.2 Repository bekijken op GitHub

Na enkele seconden verschijnt de repository ook in de browser op de GitHub-website.

- In de browser op je GitHub-pagina klik je door naar de repository `gittest`.
- Verifieer dat de bestanden `gegevens.txt` en `steden.txt` zichtbaar zijn, met inhoud volgens de recentste commit.

Op dit moment bevat GitHub dus dezelfde commits als de lokale repository.

3.3 Lokale wijzigingen maken

We zullen nu een nieuwe wijziging maken aan de lokale repository, in VS Code.

- Voeg aan `gegevens.txt` een vierde lijn toe met je favoriete kleur en sla het bestand op (`Ctrl+S`).
- Klik op `Commit`, verifieer de wijzigingen in `explorer` en vernieuw het bestand `gegevens.txt` op GitHub (`F5`).
  - Zoals verwacht is het lokale bestand aangepast.

- De favoriete kleur is echter nog niet zichtbaar op GitHub.
- In Source Control staat de laatste commit in een andere kleur dan de voorgaande commits. Twee icoontjes (concentrische cirkels en een wolkje) geven aan welke de actieve lokale commit is, en welke de recentste commit is die op GitHub staat.

De commit is lokaal opgeslagen, maar nog niet naar GitHub gestuurd.

3.4 Push uitvoeren

Om de nieuwe commit naar GitHub te sturen, gebruiken we een *push*.

- Klik in Source Control op Sync Changes.
  - Strikt genomen voert de knop Sync Changes niet enkel een push uit, maar ook eerst een Pull. Wat dat precies is, komt later aan bod (sectie 3.6).
- Vernieuw de GitHub-webpagina (F5) en verifieer dat de kleur nu wel zichtbaar is in gegevens.txt.

3.5 Wijzigingen rechtstreeks op GitHub aanbrengen

Niet alle wijzigingen moeten vanaf de eigen computer gebeuren. Een bestand kan ook rechtstreeks op GitHub worden aangepast. Later, als verschillende personen aan dezelfde repository bijdragen, zullen aanpassingen van anderen ook eerst op GitHub verschijnen.

- Open op GitHub het bestand gegevens.txt en klik op het potlood-icoon (Edit this file).
- Zet op een nieuwe lijn je favoriete hobby.
- Klik op Commit changes in GitHub en bevestig.
- Ga na of de hobby in VS Code zichtbaar is. Dat blijkt nog niet het geval te zijn.

3.6 Pull uitvoeren

GitHub bevat nu een recentere versie dan de lokale repository, om de wijzigingen van GitHub naar de eigen computer te halen, gebruiken we een *pull*.

- Ga naar Source Control (ctrl+shift+G).
- Klik op het icoontje Pull naast Graph.
- Verifieer dat de hobby nu ook lokaal zichtbaar is.

3.7 Terminologie

- *Local repository*: staat op je eigen computer.
- *Commit*: wijzigingen vastleggen in de lokale repository.
- *Remote repository*: staat online, op GitHub.
- *Push*: lokale wijzigingen naar GitHub sturen.
- *Pull*: wijzigingen van GitHub naar je eigen computer halen.

4 Werken met branches

Tot nu toe hebben we telkens rechtstreeks gewerkt op de hoofdversie van de repository. Soms willen we echter veranderingen uitproberen zonder de bestaande versie te wijzigen. Denk aan een computerprogramma waarvan versie 1 correct werkt, maar waarvan je een deel wil herschrijven om het sneller te maken. Zolang je de nieuwe versie nog niet getest hebt, en geverifieerd dat deze effectief sneller is, wil je de bestaande versie niet overschrijven maar behouden en blijven gebruiken.

Daarvoor gebruiken we een *branch*. Een branch is een alternatieve versie van dezelfde repository. Elke branch heeft zijn eigen geschiedenis van commits.

4.1 De huidige branch bekijken

Zonder het te beseffen werken we al de hele tijd in de `main`-branch.

- Helemaal onderaan links in VS Code staat de naam van de huidige branch, op dit moment heet deze "`main`". Dit is de naam van de hoofdversie van de repository. Tot nu toe werden alle commits op deze branch gemaakt.
- Verifieer dat dezelfde branchnaam op GitHub vermeld staat. De branch wordt er met hetzelfde symbool aangeduid als de `Source control`-knop in VS Code.

4.2 Een nieuwe branch maken

We zullen nu een nieuwe branch maken waarin we wijzigingen kunnen uitproberen.

- Klik in VS Code onderaan links op de branchnaam `main`.
  - Er verschijnt een optie-lijstje onder de `Command Palette` met mogelijke commando's en een overzicht van de bestaande branches, momenteel enkel de branch `main` in de lokale repository en de (momenteel identieke) branch `origin/main` in de remote repository.
- Kies voor `Create New Branch`, typ de naam "`hobbies`" in het tekstveld en druk op `Enter`.
- Door opnieuw links onderaan op de branchnaam te klikken, kan je snel wisselen tussen `hobbies` en `main`. Verifieer dat beide branches momenteel nog identiek zijn.
- Verifieer dat de nieuwe branch `hobbies` momenteel enkel lokaal bestaat en nog niet te zien is op GitHub.

4.3 Een wijziging maken op de nieuwe branch, local repository

We zullen nu een wijziging maken op de branch `hobbies`.

- Selecteer de branch `hobbies` en maak daarin een nieuw bestand `hobbies.txt`.
- Zet in het bestand een overzicht van je hobby's.
- Voer een `Commit` uit.
- Standaard worden in `Graph` enkel de commits van de huidige branch getoond. Klik naast `Graph` op het `Source Control`-icoontje, selecteer `All` en klik op OK.
- Schakel heen en weer tussen beide branches door op de branchnaam in de statusbalk te klikken.
  - Het nieuwe bestand is enkel zichtbaar in de branch `hobbies`.
  - De `Graph` toont de geschiedenis van de verschillende branches met verschillende kleuren. Labels en icoontjes geven aan waar elke branch zich momenteel bevindt. Het wolkje geeft de meest recente bekende versie van de (`main`-)branch op GitHub aan.
  - De inhoud van de werkmap in het venster `Explorer` en ook in het besturingssysteem zelf, verandert naargelang in VS Code een andere branch wordt geselecteerd.
  - De geschiedenis van commits, branches en andere repositorygegevens wordt bewaard in de verborgen submap `.git`.

4.4 Een nieuwe branch op de GitHub-repository

Nu maken we een nieuwe branch op GitHub aan en voegen er ook een extra bestand aan toe.

- Open de repository in GitHub door op `gittest` te klikken. Naast de branchnaam `main` staat de tekst "1 Branch". Klik hierop en daarna op `New Branch`. Geef de branch de naam "`vakken`". In het veld `Source branch` kies je een branch om te kopiëren (momenteel is enkel `main` beschikbaar).
- Klik op de branch `vakken`, `Add file`, `Create new file`.
- Geef het nieuwe bestand de naam `vakken.txt` en zet er de vakken in die je dit semester volgt.
- Commit de wijziging op GitHub.
  - Lokaal staan er momenteel twee branches: `main` en `hobbies`.
  - Op GitHub staan ook twee branches: `main` en `vakken`.
  - Van de branch `hobbies` ontbreekt elk spoor op GitHub en het bestaan van de branch `vakken` is onbekend in VS Code.

In wat volgt, proberen we alle branches samen te brengen. Dit kan uiteraard uitsluitend in VS Code, want GitHub heeft geen toegang tot je lokale systeem.

4.5 Fetch – online informatie ophalen

VS Code gaat niet continu na of er online aanpassingen aan de repository zijn gemaakt. Door het commando `Fetch` uit te voeren gebeurt dat wel.

- Klik op de branchnaam in VS Code, links onderaan in de statusbalk. In het lijstje bekende branches staan de lokale branches (`main` en `hobbies`) en de enige gekende remote branch (`origin/main`).
- Klik op `Fetch From All Remotes` naast `Graph`. Hierdoor gaat Git na wat er online is veranderd. Er worden op dit moment nog geen bestanden aangepast.
  - De remote branch (`origin/vakken`) verschijnt nu ook in het lijstje bekende branches als je op de branchnaam in de statusbalk klikt.

4.6 Pull – Branches van GitHub halen

Met `Fetch` gingen we enkel na *of* er online aanpassingen zijn gemaakt. Om de lokale repository effectief aan te passen aan wat er online is veranderd, gebruiken we `Pull`.

- Klik in VS Code op de branchnaam in de statusbalk en in het lijstje dat verschijnt op `Create New Branch From...`.
- Kies voor de branch `origin/vakken`.
- Geef de nieuwe lokale branch de naam `vakken`.
  - Verifieer dat nu zowel `origin/vakken` als `vakken` in de lijst bekende branches staan.
  - Schakel je over naar de branch `vakken`, dan verschijnt het bestand `vakken.txt`.
  - Het bestand `vakken.txt` verschijnt niet in de branch `hobbies` en het bestand `hobbies.txt` niet in de branch `vakken`.

4.7 Push – Branches publiceren op GitHub

Omgekeerd, om de repository op GitHub effectief aan te passen aan wat er lokaal is veranderd, gebruiken we `Push`.

- Schakel over naar de branch `hobbies`.
- Klik nu op `Publish Branch`, de branch wordt nu ook naar GitHub verzonden.
  - Op GitHub zijn nu ook de drie branches zichtbaar: `main`, `hobbies`, `vakken`.
  - In het `Graph`-venster van VS Code worden de drie branches ten opzichte van elkaar getoond, in verschillende kleuren. Bij elke branchnaam hoort nu ook een cloud-symbooltje dat de (laatst gekende) toestand van de bijhorende repository op GitHub toont.

4.8 Terminologie

- *Branch*: een alternatieve versie van de repository.
- `main`: de hoofdbranch van de repository, waarin de stabiele versie van het project wordt bewaard. De hoofdbrach kan ook een andere naam hebben, de namen `master` of `trunk` komen ook vaak voor.
- *Fetch*: nagaan welke nieuwe branches en commits op GitHub beschikbaar zijn, zonder de lokale bestanden te wijzigen
- *Pull* en *Push*: wijzigingen worden afzonderlijk voor elke branch opgehaald of gepubliceerd.

Een typische workflow is om commits niet rechtstreeks op de hoofdbranch te maken, maar eerst een nieuwe branch te maken, daar de wijzigingen uit te proberen en pas als alles correct werkt, de wijzigingen samen te voegen met de hoofdbranch. Dat proces van samenvoegen heet *merge* en komt in de volgende sectie aan bod.

5 Samenvoegen van branches: merge

We zullen in dit deel drie soorten merges uitvoeren: die van nieuwe bestanden, van wijzigingen op verschillende plaatsen in hetzelfde bestand, en van wijzigingen op dezelfde plaats in een bestand, de gevreesde *merge met conflicten*.

5.1 Lokale merge van nieuwe bestanden

Momenteel bevatten de branches `hobbies` en `vakken` elk een ander bestand dat nog niet in de branch `main` staat. We zullen beide branches samenvoegen met `main`, zodat alle bestanden er zijn samengebracht.

- Schakel over naar de *doelbranch* `main`, de branch waarin aanpassingen zullen worden overgenomen.
- Klik in `Source Control`, naast `Changes` op `More Actions`, `Branch`, `Merge`.
- Kies in de lijst voor de *bronbranch* `hobbies`, waaruit aanpassingen zullen worden overgenomen.
  - In `Explorer` verschijnt het bestand `hobbies.txt`.
  - In de `Graph` komen de paden van `main` en `hobbies` opnieuw samen.
  - Een merge resulteert eigenlijk in een commit van de aanpassingen uit de bronbranch in de doelbranch.
  - De merge is asymmetrisch, de bronbranch verandert niet.
  - De cloud-icoontjes tonen dat de merge in de remote repository nog niet is gebeurd.
- Klik op de knop `Push` naast `Graph` om de lokale aanpassingen naar GitHub te pushen.
  - In de `Graph` horen de vier branches `main`, `origin/main`, `hobbies` en `origin/hobbies` nu bij dezelfde node.

Na een merge blijven beide branches bestaan. Onmiddellijk na de merge bevatten ze dezelfde wijzigingen, maar nadien kunnen ze opnieuw onafhankelijk van elkaar evolueren. In de praktijk gebeurt verdere ontwikkeling op een secundaire branch, waarna de aanpassingen later opnieuw kunnen worden samengevoegd met de hoofdbranch `main`.

5.2 Remote merge van nieuwe bestanden

De branch `vakken` zullen we op GitHub samenvoegen met de hoofdbranch. Dit simuleert de typische situatie waarbij er twee personen betrokken zijn bij een merge, de contributor en de reviewer. De eerste voert een aantal aanpassingen door in een eigen branch, de tweede gaat na of de aanpassingen geschikt zijn om in de hoofdbranch te worden toegevoegd.

> Een *contributor* doet een bijdrage aan een repository in een eigen branch. Dit gebeurt typisch op het eigen lokale systeem, waarna de aanpassingen gepusht worden naar de GitHub-repository. Hier pasten we ter illustratie een online bestand rechtstreeks aan. Nu stelt de contributor voor om de aanpassingen in de hoofdbranch op te nemen.

- Klik in GitHub op `Pull requests` en vervolgens op `New pull request`.
- Kies in de drop-down list de ontvangende "`base`"-branch `main` en rechts de branch `vakken` waaruit de aanpassingen zullen worden gehaald. Er verschijnen twee lijsten:
  - Alle commits die er ten opzichte van de `base`-branch zijn geweest.
  - Alle bestanden (of bestandsinhouden, zie verder) die zijn gewijzigd.
- Klik op `Create pull request`, geef een beschrijving en klik opnieuw op `Create pull request`.

  In deze simulatie speel je beide rollen, maar in de praktijk zijn de volgende stappen voor de *reviewer*, die een overzicht te zien krijgt van commits en eventuele conflicten (momenteel geen) en beslist om de merge al dan niet te laten doorgaan.
- Klik op `Merge pull request`, voeg een commit message toe en bevestig met `Confirm merge`.
  - Op GitHub bevat de `main`-branch nu ook het bestand `vakken.txt`.
  - In de lokale repository is de merge nog niet gebeurd.
- Klik in VS Code naast `Graph` op de knop `Fetch From All Remotes`.
  - De `Graph` toont dat `origin/main` een nieuwere commit bevat dan `main` en dat de branch `vakken` daarbij samengevoegd wordt.
- Klik naast `Graph` op de knop `Pull`.
  - In `Explorer` zijn nu alle bestanden samengebracht in de `main`-branch.
  - In de `Graph` bevinden `main` en `origin/main` zich opnieuw op dezelfde commit.

5.3 Merge zonder conflicten

We zullen nu twee verschillende wijzigingen maken op verschillende plaatsen in hetzelfde bestand.

- Maak in de lokale repository een nieuwe branch `programmeertaal`, voeg *aan het einde* van het bestand `gegevens.txt` een nieuwe lijn toe met je favoriete programmeertaal, sla op en commit de wijziging.
- Terug in de branch `main` voeg je in het bestand `gegevens.txt` *boven je naam* je studentennummer toe. Opslaan en committen. De programmeertaal is in dit bestand nog niet te zien!
- Merge de branch `programmeertaal` met de hoofdbranch `main` (`ctrl+shift+P`, `Git: Merge...`).
  - Zowel het studentennummer als de favoriete programmeertaal verschijnen nu in het bestand `gegevens.txt` in de `main`-branch.

Git kon beide wijzigingen automatisch combineren omdat ze verschillende delen van hetzelfde bestand aanpasten. Dit werkt zolang de inhoud van eenzelfde lijn niet is aangepast.

5.4 Een merge met conflict

We creëren eerst een situatie die tot een conflict zal leiden, en zullen het daarna proberen oplossen.

- Maak een nieuwe branch `conflict`, verander in het bestand `gegevens.txt` de woonplaats in je werkelijke woonplaats, sla op en commit de wijziging.
- Terug in de branch `main` staat in `gegevens.txt` nog `Woonplaats: Gallifrey`. Verander dit in `Woonplaats: Camelot`. Dit simuleert een andere contributor die simultaan een (kennelijk opnieuw foute) aanpassing maakt. Opslaan en committen.
- Merge de branch `conflict` met `main` (`ctrl+shift+P`, `Git: Merge...`).
  - `Git` detecteert conflicterende aanpassingen en kan nu niet automatisch beslissen welke versie correct is.
  - Bekijk beide versies grondig in de editor en beslis welke actie er moet worden ondernomen.
- Klik op `Resolve merge` en dan op `accept` (of `remove`) boven de lijnen die je wil behouden (of net niet). Kies in dit geval om enkel de lijn uit de doelbranch `main` te behouden.
- Pas na het afhandelen van het conflict, kan een commit uitgevoerd worden om de merge definitief te voltooien.
- Synchroniseer nog met `GitHub` (`Sync Changes`).
  - Intussen begrijpen we dat synchroniseren, wat een pull én een push uitvoert, de lokale en de remote branch wederzijds aanpast en dus identiek maakt.

5.5 Terminologie

- *Merge commit*: een commit die het resultaat van een merge bewaart.
- *Bronbranch*: de branch waarvan de wijzigingen worden overgenomen.
- *Doelbranch*: de branch waarin de wijzigingen worden opgenomen (hierboven steeds `main`).
- *Pull request*: vraag van *contributor* om aanpassingen in doelbranch op te nemen.
- De *reviewer* inspecteert de wijzigingen vooraleer de merge uit te voeren.
- *Merge conflict*: een situatie waarin `Git` niet automatisch kan bepalen welke versie moet worden behouden.

6 Starten van een bestaande repository

Hierboven werd een repository lokaal aangemaakt, startend van een lege map, en vervolgens naar `GitHub` gepubliceerd. In de praktijk zal je echter vaak starten van een bestaande repository die zich al op GitHub bevindt: omdat je wil verder werken aan een reeds bestaande repository of omdat een teamlid de repository heeft geïnitialiseerd.

6.1 Fork – kopie van een repository van een andere eigenaar

Zelfs wanneer een repository op `GitHub` publiek beschikbaar is, kan een gebruiker die geen expliciete schrijfrechten heeft gekregen, de repository enkel bekijken en kopiëren, maar niet rechtstreeks committen of wijzigingen mergen.

Om toch aan zo'n repository te kunnen bijdragen, gebruikt GitHub zogenaamde *forks*. Een fork is een volledige kopie van een repository in een andere GitHub-account. De eigenaar van de oorspronkelijke repository behoudt de controle over de hoofdversie, terwijl de eigenaar van de fork vrij kan experimenteren in de eigen kopie.



- Open de repository `github.com/srebry/test`.
- Klik rechtsboven op `Fork`.
- Kies je eigen GitHub-account als bestemming.
- Je kan kiezen om alle branches te kopiëren door de optie `Copy the main branch only` te deselecteren (hier speelt dat geen rol).
- Bevestig.
  - In je eigen account verschijnt nu de nieuwe repository. Interessant om weten: forks van publieke repositories blijven automatisch publiek.
  - Alle bestanden en commits werden gekopieerd, en eventueel ook de branches als je dat hebt gekozen.
  - De nieuwe repository, de *fork*, blijft verwijzen naar de bron, de zogenaamde *upstream repository*, al kan je zelf geen merge uitvoeren naar deze bron.
  - Net zoals bij een branch kunnen wijzigingen via een *pull request* worden voorgesteld aan de upstream repository.
  - Verschijnen er nieuwe commits in de upstream repository, dan kunnen die worden overgenomen door in GitHub te klikken op `Sync fork`, `Update branch`.

Branches en forks lijken dus op elkaar: een branch is een alternatieve versie binnen dezelfde repository met dezelfde eigenaar. Een fork is een volledige kopie van een repository, typisch bedoeld voor een gebruiker die geen schrijfrechten heeft op de oorspronkelijke repository.

6.2 Clone – een online repository ophalen

Het lokaal kopiëren van een online repository heet *clonen*.

- Open de fork die je zonet hebt gemaakt op GitHub.
- Klik op de groene knop `Code`.
- Kopieer de HTTPS-link van de repository.
- Om een nieuw project te beginnen, open je een nieuw venster van VS Code (`ctrl+shift+N`).
- Ga naar `Source Control` en klik op `Clone Repository`
- Plak de gekopieerde URL en bevestig.
- Maak als doelmap een nieuwe map `test` in de map `C:\Projecten`.
  - Alle bestanden uit de repository verschijnen in `Explorer`.
  - De volledige geschiedenis van commits (en eventueel branches) wordt mee gekopieerd en getoond in de `Graph`.
  - Wijzigingen kunnen tussen de clone en de origin worden uitgewisseld met `Push` en `Pull`.
- Vervang je eigen r-nummer in het bestand `studenten.txt` door je voornaam. Voeg het bestand `voornaam_naam.pdf` dat je in de vorige zelfstudie LaTeX hebt gemaakt aan de juiste team-map.
- Commit en push deze aanpassingen van de clone naar jouw fork op GitHub. Maak een pull request naar de upstream repository.

Een clone is dus veel meer dan een gewone kopie van de bestanden die je via `Download ZIP` zou verkrijgen. De volledige repositorygeschiedenis wordt lokaal beschikbaar gemaakt én het blijft mogelijk om wijzigingen naar de remote repository te pushen.

6.3 Terminologie

- *Fork*: een kopie van een GitHub-repository in een andere GitHub-account.
- *Upstream repository*: de oorspronkelijke repository waarvan een fork werd gemaakt.
- *Clone*: een lokale kopie van een repository op de eigen computer.
- *Origin*: de remote repository waarvan een clone afkomstig is.

7 Werkverdeling: issues en assignees

In sectie 5 hebben we geleerd hoe een contributor wijzigingen voorstelt en een reviewer deze controleert en samenvoegt. Bij samenwerking leggen we vooraf ook vast *wat* er moet gebeuren en *wie* dat zal uitvoeren. Daarvoor gebruikt GitHub *issues* en *assignees*. Een issue beschrijft een taak, een gewenste functie of een fout; de assignee is de contributor die de taak opneemt. We oefenen eerst met één issue. In sectie 8 brengen we meerdere issues samen in een planning en in sectie 9 passen we dit toe op de teamopdracht.

7.1 Een issue en branch aanmaken

We keren terug naar de eigen oefenrepository `gittest`, met het bestand `gegevens.txt`. We beschrijven een kleine taak in een issue en voeren de wijziging uit in een aparte branch. In de praktijk verdelen we het werk over een reviewer en een contributor; in deze oefening speel je zelf beide rollen.

> De reviewer wil dat een contributor aan het bestand `gegevens.txt` de naam van de favoriete film toevoegt en voert volgende stappen uit.

- Open de repository op GitHub en klik bovenaan op `Issues`.
- Klik op `New Issue`. Geef het issue de titel “Favoriete film toevoegen” en beschrijf dat de naam van de favoriete film in `gegevens.txt` moet komen.
- Klik op `Create`.
  - Het issue verschijnt nu in de lijst met openstaande issues en krijgt automatisch een nummer. In dit voorbeeld gebruiken we nummer 1; vervang dit in de volgende stappen door het werkelijke nummer van jouw issue.
- Open het issue dat net is aangemaakt.
- Klik op de tekst `Create a branch` in de rechter kolom onder `Development`.
- Maak de branch aan vanaf `main`. Geef de branch de naam `1-film`, met het issue-nummer en een korte omschrijving.
  - De branch bestaat momenteel enkel op GitHub.
  - GitHub voorziet Git-commando’s om de nieuwe branch naar de lokale repository te pullen, maar de werkwijze vanuit VS Code zoals hierboven werkt even goed.

7.2 Een assignee toekennen

Een issue kan aan één of meerdere personen worden toegewezen.

- Open opnieuw het zonet aangemaakte issue.
- Klik op het tandwiel naast `Assignees` in de rechter kolom.
- Selecteer een of meerdere contributors van de repository (voorlopig is dat enkel jijzelf).
  - Het issues-overzicht toont op elk moment welke taken nog openstaan.

- In de meest rechtse kolom zie je wie verantwoordelijk is voor een taak en welke taken nog niet zijn toegekend.
- De filter `Assignees` laat toe de taken van een bepaalde contributor (bijvoorbeeld jezelf) te tonen.

7.3 De wijziging uitvoeren

Deze kleine aanpassing kunnen we rechtstreeks op GitHub doorvoeren. Grotere issues vergen de fetch-pull-commit-push-cyclus uit deel 4.

De contributor die het issue zal opvolgen, voert volgende stappen uit.

- Open op GitHub in de branch `1-film` het bestand `gegevens.txt`. Voeg een nieuwe lijn toe met daarop de naam van je favoriete film en sla het bestand op.
- Maak een commit. De gevraagde wijziging staat nu in de branch `1-film`, maar nog niet in `main`. Het issue blijft daarom open.
- Maak een pull request van `1-film` naar `main`, zoals in sectie 5. Controleer dat het issue bij `Development` is gekoppeld. Je kunt de koppeling ook leggen door `Closes #1` in de beschrijving op te nemen, met het juiste issue-nummer.

7.4 Het issue sluiten

De reviewer controleert of de bijdrage aan de taakbeschrijving voldoet. Het issue wordt pas gesloten wanneer de gecontroleerde wijziging in `main` is opgenomen.

- De reviewer behandelt de pull request. Zijn er nog verbeteringen nodig, dan past de contributor dezelfde branch aan en volgt een nieuwe controle. Na goedkeuring klikt de reviewer op `Merge pull request` en vervolgens op `Confirm merge` om de bijdrage naar `main` te mergen.
  - Omdat de pull request aan het issue is gekoppeld en naar de standaardbranch `main` wordt gemerged, sluit GitHub het issue automatisch. Je vindt het terug bij de `Closed issues`.
  - Blijkt de merge toch niet afdoende, dan kan het issue opnieuw worden geopend: GitHub `Issues`, `Closed`, klik op het onterecht gesloten issue, `Reopen issue`.
  - Is de taak op een andere manier afgewerkt, dan kan het issue manueel worden afgesloten. Beschrijf eerst in een reactie hoe de taak is afgewerkt en klik vervolgens op `Close issue`.
  - Via de gesloten issues kun je achteraf nagaan wie een taak heeft uitgevoerd en welke wijzigingen erbij horen.

Een merge naar een andere branch sluit het issue niet op deze manier. Bij de teamopdracht komt een bijdrage eerst in een deelbranch terecht; pas na opname in `main` is de taak volledig afgewerkt (sectie 9).

7.5 Terminologie

- *Issue*: een probleem, taak of vraag die moet worden opgevolgd. Issues kunnen erg uiteenlopend zijn, ze kunnen een fout (*bug*) in een bestaand document signaleren, of vragen om een extra functie (*feature request*).
- *Assignee*: de contributor die is toegekend aan het oplossen van een specifiek issue.
- *Open issue*: een issue waarvoor nog geen oplossing beschikbaar is.
- *Closed issue*: een issue dat werd opgelost of afgewerkt.

8 Projectplanning met GitHub Projects

Met de issues en assignees uit sectie 7 leggen we vast wat er moet gebeuren en wie een taak uitvoert. Om ook de voortgang en de deadlines te overzien, gebruiken we *GitHub Projects*. Een project kan de issues voorstellen als kaarten op een *kanbanbord*: de kolom waarin een kaart staat, geeft de status van de taak aan. Het bord is dus geen afzonderlijke takenlijst, maar een andere weergave van de bestaande issues.

In deze sectie oefen je het instellen en gebruiken van een projectbord bij je eigen repository `gittest`. Voor de teamopdracht staat het gezamenlijke bord al klaar. Hoe je daarmee werkt, lees je in sectie 9.

8.1 Een project aanmaken en delen

Maak voor deze oefening een project onder je eigen GitHub-account aan.

1. Open op GitHub je profiel en kies `Projects`, `New project`. Kies onder `Start from scratch` voor `Board`, geef het project een herkenbare naam, bijvoorbeeld “Planning – gittest”, en klik op `Create project`.
2. Open de repository en kies `Projects`, `Link a project`. Selecteer het aangemaakte project.
3. Open in het project het menu rechtsboven en kies `Settings`, `Manage access`. Werk je samen met anderen, nodig hen dan uit met de rol `Write`, zodat zij het bord kunnen aanpassen.

Toegang tot het project en toegang tot de repository worden afzonderlijk beheerd. Controleer bij samenwerking dus dat iedereen toegang heeft tot beide.

8.2 Issues aan het project toevoegen

Open een bestaand issue in de repository. Klik in de rechterkolom op `Projects` en selecteer het project. Het issue verschijnt nu ook als kaart op het bord. Je hoeft de taakbeschrijving dus niet opnieuw in te voeren. Herhaal dit voor de andere taken.

Gebruik voor deze werkwijze de issues als taakkaarten. Maak niet nog eens een afzonderlijke taak aan voor de pull request waarmee hetzelfde issue wordt opgelost.

8.3 De voortgang bijhouden

De bordkolommen worden bepaald door het veld `Status`. Open de projectinstellingen en pas de opties van dit veld aan. Controleer op het bord bij `View`, `Column field` dat `Status` geselecteerd is. Geef de kolommen namen die bij de werkwijze passen, zoals `Te doen`, `Bezig` en `Afgewerkt`. De uitgebreidere statussen voor de teamopdracht staan in sectie 9.4.

Sleep een kaart naar een andere kolom om de status te wijzigen. Dit is alleen een aanpassing van de planning: het verplaatsen van een kaart vervangt geen review of merge.

Via het projectmenu `Workflows` kun je instellen dat gesloten issues automatisch de status `Afgewerkt` krijgen. Kies daarbij de statusnaam die je op jouw bord gebruikt. Het sluiten van een issue en het wijzigen van de bordstatus blijven afzonderlijke handelingen, tenzij je hiervoor een automatisering instelt.

8.4 Deadlines toevoegen en zichtbaar maken

We voegen een datumveld toe om voor elke taak een deadline vast te leggen.

1. Klik in het project op `View` en kies onder `Layout` voor `Table`.
2. Klik rechts van de kolomkoppen op de knop om een veld toe te voegen en kies `New field`. Geef het veld de naam `Deadline`, selecteer het type `Date` en klik op `Save`.
3. Vul in de nieuwe kolom voor elk gepland issue een datum in. Je kunt deze intypen of selecteren met de kalender.

Schakel terug naar `Board`. Kies `View`, `Fields` en maak `Assignees` en `Deadline` zichtbaar op de kaarten. Via `View`, `Sort`, `Deadline` kun je de kaarten ook op deadline sorteren.

Om alleen openstaande issues te zien waarvan de deadline verstreken is, typ je in de filterbalk:

```
is:issue is:open deadline:<@today
```

Hierbij staat `@today` voor de huidige dag. Verwijder de filter om opnieuw alle taken te zien.

Het bord toont nu per taak wie eraan werkt, wat de status is en wanneer de taak klaar moet zijn. In sectie 9 verbinden we deze planning met de branches, reviews en teamvergaderingen van de teamopdracht.

9 De teamopdracht: repository, planning en workflow

Voor de teamopdracht combineren we de werkwijzen uit de vorige secties. De repository bevat de bestanden en hun geschiedenis, issues beschrijven de taken en het projectbord toont de planning. Een bijdrage doorloopt steeds dezelfde stappen: afspreken, uitwerken, controleren en samenvoegen.

9.1 Starten in je teamrepository

Voor je team staat een gezamenlijke repository `teamopdrachtX` klaar, met `X` de letter van je team, bijvoorbeeld `teamopdrachtA`. Je werkt daarin samen met alle teamleden. De teamrepository is al een fork.

Accepteer eerst de uitnodiging voor de onderwijsorganisatie. Open via de link van de begeleiding je teamrepository en controleer onder `Projects` of je ook het gelijknamige projectbord kunt openen. Lukt dat niet, vraag dan hulp aan de begeleiding. Maak vervolgens een lokale clone op je eigen computer, zoals geoefend bij het clonen van een online repository. De doorlopende teamopgave staat in `opgave/FAteamopdracht2026.pdf`; sessie 4 daarin beschrijft de Git-sessie. De `README.md` in de repository bevat links naar de opgave, deze handleiding en de basisteksten.

9.2 Bestanden uit sessies 1 tot en met 3

Breng het werk uit de eerste drie sessies samen in je teamrepository. Behoud de bestandsnamen uit die opgaven. Voor het werken met Git spreken we nu de volgende indeling af:

`opgave/` Lees hier het doorlopende opgavedocument. Nieuwe sessies worden aan dezelfde PDF toegevoegd. Haal de bijgewerkte `main` op om de nieuwste versie te lezen.

`vergaderingen/` Bewaar hier de start- en eindverslagen, bijvoorbeeld `teamXstartvergadering02.txt`, `teamXeindvergadering02.txt` en de overeenkomstige bestanden van sessie 3. De notulist werkt hiervoor in `1-vergaderingen`.

`oefeningen/` Verzamel de foto's van de uitwerkingen samen in deze map, met één oefening per foto en de naam `teamXopdrachtXX-auteur.jpg`, zoals gevraagd in sessie 3. Vervang `X` door de teamletter, `XX` door het oefeningnummer en `auteur` door de auteur van de uitwerking. Behoud verschillende oplossingen en noteer bij onvolledige of foutieve uitwerkingen wat nog ontbreekt of misloopt.

`bronnen/` Hier staan de aangeboden basisteksten en deze Git-handleiding. Verzamel eigen aanvullend geraadpleegde bronnen in `bronnen/aanvullend/`, met links en bibliografische gegevens zoals gevraagd in sessie 2.

Bewaar `teamXrolverdeling.txt` en `teamXbegrippenlijst.xlsx` in de hoofdmap. Vervang ook hier `X` door je teamletter. De notulist brengt de rolverdeling samen met de vergaderverslagen in via `1-vergaderingen`. Voor de begrippenlijst en aanvullende bronnen spreek je een verantwoordelijke af en maak je een eigen issue. Werk die bij via een issue-branch vanaf `main`, laat een ander teamlid de pull request controleren en laat de teamleider die naar `main` mergen. Stem wijzigingen aan hetzelfde Excel-bestand vooraf af en werk om de beurt, zodat je elkaars aanpassingen kunt overnemen.

Een map ordent bestanden; een branch bevat een versie van de hele repository. De map `oefeningen/` en de branch `2-oefeningen` hebben dus verschillende functies. Deze Git-afspraken vervangen de gevraagde indieningen via Toledo niet: maak voor sessie 3 ook het gevraagde `teamXoefeningen.zip` van de foto's en dien de bestanden in volgens de sessieopgave.

9.3 Branches en verantwoordelijkheden

We onderscheiden twee soorten branches:

- *Langlevende branches* verzamelen werk gedurende een langere periode. De hoofdbranch `main` bevat de goedgekeurde eindproducten: opgeloste oefeningen, Python-scripts, het eindverslag en de poster. In deelbranches zoals `1-vergaderingen` en `2-oefeningen` brengen we bijdragen voor een onderdeel samen. Beide branches staan al klaar. Gebruik `1-vergaderingen` voor de vergaderverslagen en de rolverdeling, en `2-oefeningen` voor de oefeningen. De indeling voor latere deelopdrachten spreken we af wanneer die opgaven beschikbaar zijn. Een nieuwe deelbranch vertrekt van de bijgewerkte `main`.

- Een *feature branch* dient voor één afgebakende taak en vertrekt van de bijbehorende deelbranch. Een branch `23-oefening01` voor issue 23 vertrekt bijvoorbeeld van `2-oefeningen`. Andere voorbeelden zijn een branch voor een fout in een Python-script, voor spellingscontrole of voor een posterfiguur. Na de merge naar de deelbranch kan de feature branch worden verwijderd.

De contributor werkt een taak uit in een feature branch, ook wel *issue-branch* genoemd. De taakverantwoordelijke wijst per pull request een ander teamlid als reviewer aan, volgt de feedback op en mergt de gecontroleerde bijdragen naar de deelbranch. Voor de oefeningen is dat de verantwoordelijke oefeningen. Niemand keurt de eigen bijdrage goed. De teamleider controleert vervolgens het volledige onderdeel en neemt het via een pull request op in `main`.

Voor vergaderverslagen bereidt de verslaggever het verslag voor in `1-vergaderingen`. De teamleider mergt het pas naar `main` nadat het team het verslag heeft goedgekeurd, meestal op de volgende vergadering.

9.4 Het projectbord en de planning afspreken

Open via `Projects` het klaargezette bord `teamopdrachtX`. Gebruik dit bord samen met je team gedurende de volledige opdracht. Je kunt taken toevoegen, jezelf als assignee instellen en de planning bijwerken. Het aanmaken en instellen van een eigen bord heb je geoefend in sectie 8.

Gebruik het veld `Status` volgens deze afspraken:

Te doen: De taak is beschreven, maar nog niet gestart.

Bezig: De assignee werkt aan de taak.

Ter controle: De contributor heeft de bijdrage uitgewerkt en een pull request aangemaakt. De reviewer moet de wijzigingen nog controleren of opnieuw beoordelen.

Nog te integreren: De gecontroleerde bijdrage staat in de deelbranch, bijvoorbeeld `2-oefeningen`. Ze is nog niet in `main` opgenomen.

Afgewerkt: De bijdrage is gecontroleerd en opgenomen in `main`.

Op de kaarten zie je bij `Assignees` wie de taak opneemt en bij `Deadline` wanneer ze klaar moet zijn. Vul die einddatum in zoals geoefend in sectie 8.4. Een deadline betekent hier dat de taak volledig afgewerkt moet zijn, inclusief controle en opname in `main`. Voorzie dus tijd voor feedback, verbeteringen en de integratie van de deelbranch.

De vrijdagsessie is een tussenmijlpaal voor het inbrengen en reviewen van bestaand werk. De integratie in `main` kan later volgen. Noteer een eerdere upload- of reviewafspraak in het issue; vul bij `Deadline` de afgesproken datum voor de volledige afronding in. De begeleiding en het team spreken die einddatum samen af.

Het project bevat drie klaargezette weergaven: `Planning` toont de werkverdeling als kanbanbord, `Deadlines` toont de taken en einddatums in een tabel en `Achterstallig` toont openstaande issues waarvan de deadline verstreken is.

9.5 Taken vastleggen en verdelen

In je teamrepository staan de 13 issues `oefening01` tot en met `oefening13` al klaar. Je vindt ze ook op het projectbord onder `Te doen`. Ze verwijzen naar Opdrachten 1–13 uit *Orthogonale functies en Fourierbenaderingen*. Gebruik deze bestaande issues; het issue-nummer hoeft niet gelijk te zijn aan het opdrachtnummer. De tekst met deze opdrachten staat als `OrthogonaleFunctiesFourierTransformaties.pdf` in `bronnen/`.

Verdeel tijdens de teamvergadering de bijdragen en spreek de deadlines af. Voor nieuwe, afzonderlijke taken maakt de taakverantwoordelijke een issue aan met het verwachte resultaat en de bijbehorende deelbranch. Voeg zo'n nieuw issue ook toe aan het projectbord met status `Te doen`.

Spreek af wie welke taak opneemt. Stel jezelf in als assignee wanneer je aan een taak begint en verplaats de kaart naar `Bezig`. Werk bij voorkeur aan één taak tegelijk, zodat de taakverdeling overzichtelijk blijft. Nieuwe taken, zoals een ontdekte fout of een te herschrijven tekst, krijgen een eigen issue. Maak voor de bijbehorende pull request geen tweede taakkaart aan.

Werken meerdere leden aan dezelfde oefening, dan kunnen zij hetzelfde issue gebruiken met meerdere assignees en afzonderlijke pull requests. De kaart krijgt pas `Ter controle` wanneer alle afgesproken bijdragen klaarstaan voor review, en pas `Nog te integreren` wanneer alle vereiste bijdragen gecontroleerd in de deelbranch staan. Leg in het issue vast welke bijdragen nog ontbreken.

9.6 Opdracht voor de vrijdagsessie: bestaand werk inbrengen

Tijdens deze sessie brengt ieder teamlid eigen materiaal van de afgelopen weken in. We oefenen tegelijk het verdelen, reviewen en samenvoegen van werk. De notulist brengt ook de vergaderverslagen en de rolverdeling in; spreek af wie de bestaande begrippenlijst en aanvullende bronnen toevoegt volgens sectie 9.2. Voor de oefeningen volg je deze stappen.

1. Help elkaar eerst de basisoefeningen uit deze handleiding tot en met de pull request af te werken. Gebruik daarvoor de oefenrepository.

2. Controleer je toegang tot de teamrepository en het projectbord, maak een lokale clone en spreek af wie de verantwoordelijke oefeningen en de teamleider zijn.

3. Open het issue voor jouw oefening, wijs het aan jezelf toe en zet het op `Bezig`. Haal `2-oefeningen` op en maak daarvan een issue-branch. Gebruik het werkelijke issue-nummer, de oefening en je GitHub-naam, bijvoorbeeld `23-oefening01-jenaam`.

4. Voeg je foto's rechtstreeks toe in `oefeningen/`, met de naamgeving uit sessie 3, bijvoorbeeld `teamAopdracht01-jenaam.jpg`. Houd verschillende uitwerkingen herkenbaar aan hun auteursnaam. Voeg eventuele bronbestanden en bijlagen bij de betreffende oefening toe en licht toe wat ze bevatten. Maak per oefening een afzonderlijke branch en pull request.

5. Commit en push zelf. Maak een pull request naar `2-oefeningen` in je eigen teamrepository en vermeld `Zie #23`, met het juiste issue-nummer. Beschrijf wat je inbrengt, met wie het is gemaakt en wat nog ontbreekt.

6. De verantwoordelijke oefeningen wijst een andere reviewer aan. Doorloop de controle en verbeteringen uit de volgende paragraaf. Goedgekeurde bijdragen worden naar `2-oefeningen` gemerged. Het issue blijft open totdat het werk ook in `main` staat.

Bewaar de verschillende oplossingen van je teamgenoten, zoals gevraagd in sessie 3. Noteer bij onvolledige of foutieve oplossingen wat nog ontbreekt of misloopt. De verantwoordelijke oefeningen houdt bij welke uitwerkingen gecontroleerd zijn en welke verbeteringen nog nodig zijn. Lees als teamlid ook de andere oplossingen, zodat je alle oefeningen begrijpt. De volledige bestandsafspraken staan in `oefeningen/README.md`.

Controleer aan het einde van de sessie of je het afgesproken materiaal hebt gepusht en een pull request hebt aangemaakt. Goedgekeurde bijdragen staan in `2-oefeningen`. Noteer resterende feedback, ontbrekend werk en afspraken voor de volgende controle in de pull requests en issues. Werk het bord bij. Het volledige oefeningenpakket hoeft tijdens deze sessie nog niet naar `main` te worden gemerged.

9.7 Van bijdrage tot afgewerkte taak

Voor een oefening verloopt de samenwerking als volgt:

1. Haal als contributor de recentste versie van `2-oefeningen` op en maak daarvan een feature branch, bijvoorbeeld `23-oefening01`. Werk de taak uit, maak regelmatig een commit en push de wijzigingen naar de teamrepository.

2. Maak zodra je bijdrage klaar is voor controle een pull request naar `2-oefeningen`. Vermeld het issue-nummer in de beschrijving, bijvoorbeeld `Zie #23`, en verplaats de kaart naar `Ter controle`.

3. Ben je aangewezen als reviewer, controleer dan of alle gevraagde onderdelen aanwezig zijn, of de redeneringen en resultaten kloppen en of eventuele code met de beschreven stappen werkt. Dien je review in met goedkeuring of concrete verbeterpunten. Als contributor verwerk je gevraagde verbeteringen in dezelfde feature branch en push je opnieuw. Zo wordt de pull request bijgewerkt. Zet de kaart tijdens het herwerken op `Bezig` en zodra je opnieuw review vraagt op `Ter controle`.

4. Na goedkeuring mergt de taakverantwoordelijke de bijdrage naar `2-oefeningen`. De kaart krijgt de status `Nog te integreren`. Het issue blijft open.

5. Wanneer het onderdeel klaar is, maakt de taakverantwoordelijke een pull request van `2-oefeningen` naar `main`. Vermeld welke issues daarmee worden afgewerkt. De teamleider controleert het geheel en voert na goedkeuring de merge uit.

6. Sluit de bijbehorende issues pas na deze laatste merge. Zet bijvoorbeeld `Closes #23` in de beschrijving van de pull request naar `main` om het issue bij de merge automatisch te sluiten. Controleer daarna of de kaarten op `Afgewerkt` staan.

Een sluitinstructie zoals `Closes #23` in de beschrijving van een pull request werkt alleen wanneer die pull request de standaardbranch `main` als doel heeft. Bij de eerste merge naar `2-oefeningen` gebruik je daarom een gewone verwijzing naar het issue. Controleer bij de uiteindelijke merge naar `main` welke taken daadwerkelijk afgewerkt zijn en sluit alleen die issues.

9.8 Wanneer gaat het oefeningenpakket naar main?

De verantwoordelijke oefeningen stelt het integratieverzoek op zodra alle vereiste oefeningen en bijlagen aanwezig zijn, de reviews en feedback zijn afgehandeld en de vereiste uitwerkingen gecontroleerd zijn. Bewaar ook alternatieve uitwerkingen en licht eventuele fouten of onvolledigheden toe. De teamleider beoordeelt het volledige pakket en voert de merge naar `main` uit. Dat kan enkele weken na de vrijdagsessie zijn. Tot dan blijven de gecontroleerde oefeningen `Nog te integreren`. Na de integratie kunnen latere correcties via nieuwe issues en pull requests worden toegevoegd.

9.9 Nieuwe opgaven ophalen

Wanneer nieuwe opgaven in je teamrepository beschikbaar zijn, haal je de bijgewerkte `main`-branch met `Pull` naar je lokale clone. De verantwoordelijke oefeningen neemt relevante updates via een pull request van `main` naar `2-oefeningen` over. Werk je in een issue-branch, haal dan daarna de bijgewerkte `2-oefeningen` op en merge die waar nodig in je eigen branch. Los eventuele conflicten samen met de verantwoordelijke op voordat je verderwerkt. Voor andere deelbranches geldt dezelfde werkwijze.

9.10 De voortgang samen opvolgen

Open het projectbord bij iedere teamvergadering. Bespreek eerst taken die vastlopen, op controle wachten of hun deadline naderen. Bekijk ook welke bijdragen nog vanuit een deelbranch naar `main` moeten worden overgebracht. Verdeel daarna het nieuwe werk en leg de deadlines vast.

Werk het bord ook tussen de vergaderingen bij. Loop je vast, beschrijf dan de blokkade in een reactie bij het bestaande issue, zodat duidelijk is welke hulp nodig is. Maak een nieuw issue als er een afzonderlijke taak bijkomt. Push je werk regelmatig en in elk geval aan het einde van een teamsessie: alleen gepushte commits zijn zichtbaar voor de andere teamleden en de begeleiding.

9.11 Je bijdrage zichtbaar maken

Beschrijf in je commits en pull requests wat je hebt uitgewerkt en welke keuzes je hebt gemaakt. Bekijk tijdens een teamvergadering samen de wijzigingen, issues en ingediende reviews. Zo kun je bespreken wie welke taken heeft uitgevoerd, wie feedback gaf en waar nog hulp nodig is. Ook overleg, onderzoek en samen programmeren horen bij je bijdrage.

Breng je bestaand materiaal in, vermeld dan in de pull request wie eraan heeft gewerkt. De eerste upload laat zien wie bestanden heeft toegevoegd, maar vertelt niet hoe het eerdere werk tot stand kwam. Veel commits of gewijzigde regels betekenen op zichzelf niet dat je meer hebt bijgedragen.

Je begeleiding volgt je voortgang via je gepushte werk, issues, pull requests en reviews, ook zolang dat werk nog niet in `main` staat. De voortgangsrapporten zijn alleen voor de begeleiding toegankelijk. Geef je GitHub-gebruikersnaam en het voor commits gebruikte e-mailadres door, zodat je werk aan je account kan worden gekoppeld. Het e-mailadres hoeft niet publiek te staan.

Besluit

Git en GitHub vormen vandaag de standaard voor versiebeheer en samenwerking in professionele projecten. Ze worden gebruikt van zeer kleine projecten door één gebruiker, tot grote software-, onderzoeks- en ingenieursprojecten waaraan honderden of zelfs duizenden personen tegelijk bijdragen.

Bij een eerste kennismaking lijkt Git vaak complex. De begrippenlijst is ellenlang – *repository*, *commit*, *branch*, *merge*, *issue*, *pull request*, *contributor* en *reviewer* – en deze uitgebreide terminologie is niet altijd intuïtief. Die begrippen zijn echter noodzakelijk om de onderliggende ideeën te begrijpen en om efficiënt te kunnen samenwerken in grotere projecten.

Door Git en GitHub consequent te gebruiken ontstaan verschillende voordelen. Dankzij commits en repositories gaat informatie niet verloren wanneer bestanden per ongeluk worden verwijderd of onbedoeld worden aangepast. Nieuwe ideeën kunnen eerst worden ontwikkeld in afzonderlijke branches zonder de stabiele versie van het project in gevaar te brengen en meerdere personen kunnen gelijktijdig aan verschillende onderdelen werken. Tot slot biedt werken met Git en GitHub niet alleen technische hulpmiddelen voor versiebeheer, maar ondersteunt ook de organisatie van het project zelf. Issues, assignees, branches en pull requests maken het teamwerk zichtbaar.

Git en GitHub zijn dus veel meer dan een systeem om bestanden op te slaan. Ze vormen een hulpmiddel om kennis, documenten, code en verantwoordelijkheden op een gestructureerde manier te beheren.